
GAARA-II

Documentação Técnica Completa e Arquitetura do Sistema

Plataforma Inteligente de Vigilância Epidemiológica, Análise de Séries Temporais,
Alerta Precoce e Simulação de Arboviroses no Brasil

Projeto: GAARA-II (Dashboard Inteligente de Arboviroses)

Versão Atual: 2.0.0 (Python FastAPI + Angular 17+)

Referência Científica: Projeto de Pesquisa (Subprojetos 1 e 2)

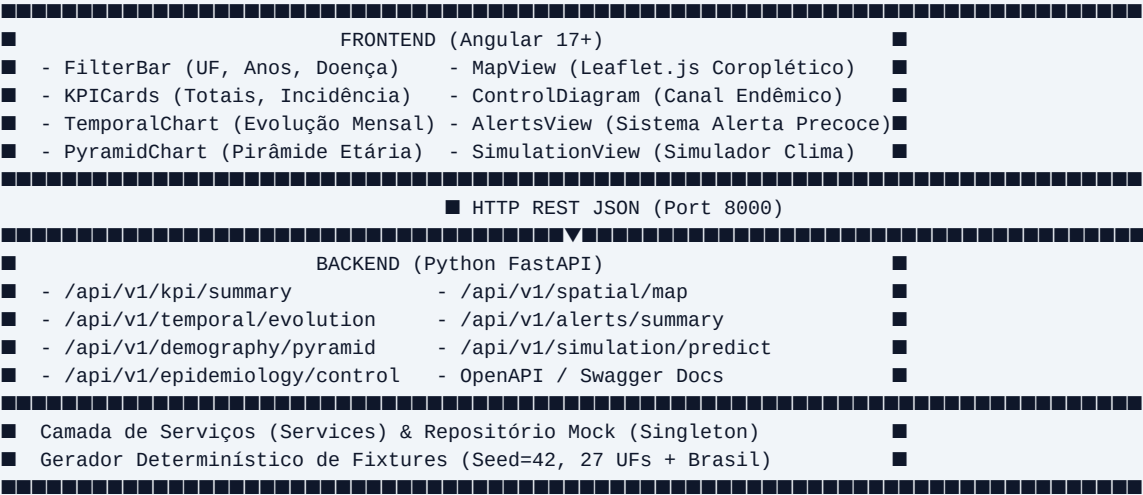
Base Legada de Comparação: Scientific_Initiation_Dashboard (R / Shiny)

Data de Elaboração: Agosto de 2026

Status de Diagnóstico: Dengue Implementado | Plano de Expansão Multi-Arboviroses Estruturado

1. Visão Geral e Arquitetura do GAARA-II

O **GAARA-II** é uma plataforma moderna desenvolvida para transformar a vigilância epidemiológica de arboviroses no Brasil. O sistema substitui e aprimora a versão legada monolítica em R/Shiny (*Scientific_Initiation_Dashboard*), adotando uma arquitetura **totalmente desacoplada**, resiliente e escalável composta por um backend REST em **Python (FastAPI)** e um frontend SPA em **Angular 17+**.



2. Análise Detalhada dos Componentes de Backend

A pasta backend/app/ foi estruturada de forma modular seguindo as melhores práticas de Clean Architecture e injeção de dependências do FastAPI:

Módulo / Arquivo	Responsabilidade e Lógica Epidemiológica
main.py	Inicialização do FastAPI, configuração de CORS para localhost:4200, inclusão das rotas da API v1 e endpoint /health.
core/config.py	Definição de variáveis de ambiente, nomes de projeto, prefixos REST e flag USE MOCK DATA=True via Pydantic Settings.
core/constants.py	Mapeamento das 27 UFs, populações estimadas 2024, regiões brasileiras, faixas etárias padrão (0-4 a 80+) e faixas de incidência por 100k hab.
mocks/mock_data_generator.py	Gerador determinístico baseado em função periódica (seno) com seed=42. Produz séries mensais (2014-2026), canais endêmicos (Q1, Mediana, Q3), pirâmides e alertas com sazonalidade realista.
repositories/mock_repository.py	Padrão Singleton que carrega os dados gerados uma única vez em memória. Aplica filtros de UF, períodos e proporções dinâmicas sem recomputar do zero.
services/kpi_service.py	Calcula total de notificados, confirmados, óbitos (suspeitos e confirmados), taxa de letalidade (%) e coeficiente de incidência por 100 mil habitantes.
services/temporal_service.py	Agrega dados históricos mensais de casos notificados e confirmados para a UF selecionada ou agregado Brasil.

services/demography_service.py	Estrutura a distribuição demográfica cruzando 10 faixas etárias com os sexos Masculino e Feminino.
services/epidemiology_service.py	Calcula o Canal Endêmico semanal (Q1 a Q3), mediana histórica e sobrepõe a incidência recente com sinalizador booleano em_alerta.
services/spatial_service.py	Fornecer dados espaciais com classificação de risco em 5 faixas cromáticas para os 27 estados e municípios com coordenadas geográficas.
services/alerts_service.py	Filtra e sumariza notificações ativas por severidade (Crítico, Alto, Moderado) com planos de ação e variação percentual sobre o limiar Q3.
services/simulation_service.py	Executa modelo de elasticidade bioclimática calculando o impacto de variações térmicas (+8%/°C), pluviométricas e de umidade sobre a curva de casos.

3. Análise Detalhada dos Componentes de Frontend (Angular 17+)

O frontend foi construído utilizando o paradigma de **Standalone Components** do Angular 17, eliminando módulos desnecessários e garantindo máxima velocidade de carregamento e manutenibilidade:

Componente / Serviço	Função na Interface e Integração
EpidemiologyApiService	Cliente HTTP centralizado que consome a API REST com mecanismo de fallback automático offline via operadores RxJS (catchError).
FilterStateService	Estado reativo global baseado em BehaviorSubject. Notifica todos os componentes instantaneamente quando UF ou período é alterado.
DashboardComponent	Componente container que dispara 5 requisições paralelas e distribui os dados para os cards, gráficos temporais, pirâmides, diagramas e mapas.
KpiCardsComponent	Apresenta os 4 cards de KPIs: Total de Casos, Confirmados, Óbitos e Taxa de Incidência/Letalidade com estilização moderna.
TemporalChartComponent	Visualização combinada (barras de notificados + linha de confirmados) ao longo dos meses selecionados.
ControlDiagramComponent	Gráfico do Canal Endêmico (faixa Q1-Q3 com mediana tracejada) e sobreposição da incidência atual, destacando semanas em alerta.
PyramidChartComponent	Gráfico de barras bidirecional espelhado com distribuição de casos por sexo e faixas etárias padronizadas.
MapViewComponent & DataTable	Mapa coroplético interativo Leaflet.js dos estados brasileiros com faixas de risco e tabela de dados territoriais detalhada.
AlertsViewComponent	Módulo completo de Alertas Precoces com cards categorizados, badges animados de severidade e listas de ações imediatas de controle vetorial.
SimulationViewComponent	Simulador interativo de cenários climáticos com sliders de temperatura, chuva e umidade, renderizando curvas SVG em tempo real.

4. Comparativo: Projeto Legado (R/Shiny) vs. GAARA-II

A migração do projeto legado `Scientific_Initiation_Dashboard` (desenvolvido em R/Shiny) para o **GAARA-II** representou um salto qualitativo em performance, segurança, desacoplamento e experiência de usuário:

Critério de Comparação	Legado (<code>Scientific_Initiation_Dashboard</code>)	GAARA-II (Nova Arquitetura)
Arquitetura	Monolítica em R/Shiny. Processamento e renderização centralizados no servidor R.	Desacoplada. API REST FastAPI independente e SPA Angular client-side.
Escalabilidade	Limitada pelo pool de conexões e threads do processo Shiny Server.	Alta escalabilidade horizontal com Uvicorn, Nginx e containerização Docker.
Gráficos e UI	Gráficos gerados como imagens ou widgets R (ggplot2), com recarregamento lento.	Gráficos interativos vetoriais com animações suaves e renderização D3/SVG.
Alerta Precoce	Inexistente no legado. Usuário precisava interpretar o gráfico manualmente.	Módulo dedicado com classificação de risco e emissão de ações recomendadas.
Simulação Climática	Inexistente. Apenas visualização histórica passiva.	Simulador interativo de anomalias térmicas e pluviométricas integrado.
Tipagem e Robustez	Scripts R dinâmicos sem validação estrita de contratos de dados.	Tipagem estrita ponta a ponta com TypeScript e Pydantic v2.
Dark / Light Mode	Código comentado e incompleto em <code>1grafico.R</code> .	Design System completo com tokens CSS/SCSS e alternador instantâneo.

5. Matriz de Conformidade com o Projeto de Pesquisa (PDF)

Confrontando o código implementado com os requisitos estipulados no documento '**Projeto de Pesquisa (2).pdf**' (organizado em Subprojeto 1 — Modelagem e Banco, e Subprojeto 2 — Dashboard e Alertas):

Requisito do Projeto de Pesquisa	Subprojeto	Status no GAARA-II	Diagnóstico e Próximos Passos
Dashboard interativo com filtros e KPIs	Subprojeto 2	IMPLEMENTADO	Atende integralmente aos requisitos de visualização dinâmica.
Canal Endêmico e Diagrama de Controle	Subprojeto 2	IMPLEMENTADO	Quartis Q1, Mediana e Q3 com alerta de ultrapassagem calculados.
Sistema de Alerta Precoce regionalizado	Subprojeto 2	IMPLEMENTADO	Interface e lógica de regras prontas; falta envio de e-mails automáticos.
Simulação de cenários climáticos	Subprojeto 1 / 2	IMPLEMENTADO	Mecanismo paramétrico funcional com fórmula de elasticidade vetorial.
Banco de Dados PostgreSQL + PostGIS	Subprojeto 1	PENDENTE	Atualmente usa MockRepository. Schema dimensional rascunhado.
Pipeline ETL automático (SINAN/DataSUS)	Subprojeto 1	PENDENTE	Tradução do script R <code>sinan_download_data.R</code> para Python necessária.
Suite de Modelos de IA (SARIMA, GLMM, LSTM)	Subprojeto 1	ISOLADO (STUB)	Contratos REST definidos aguardando calibração dos pesquisadores.

Expansão para Múltiplas Arboviroses	Subprojeto 1 / 2	PENDENTE	Código atual opera 100% focado em dengue. Plano de expansão detalhado na Seção 6.
-------------------------------------	------------------	-----------------	---

6. Diagnóstico e Plano de Expansão para Múltiplas Arboviroses

Confirmação de Diagnóstico: A análise minuciosa do código confirmou que o GAARA-II opera **exclusivamente com dados e regras de Dengue** (vide `main.py`, `config.py` e curvas sazonais de `mock_data_generator.py`). Não há suporte atual a Chikungunya, Zika ou Febre Amarela nos schemas ou filtros.

Para transformar o GAARA-II em um monitor multivariado completo de arboviroses conforme o Projeto de Pesquisa, o seguinte plano de engenharia deve ser executado:

- **Fase 1 — Domínio das Arboviroses (Backend):** Adicionar o enum `ArbovirusEnum` (`DENV`, `CHIKV`, `ZIKV`, `YFV`) e dicionário de configuração com parâmetros sazonais e pesos anuais específicos para cada agravo em `core/constants.py`.
- **Fase 2 — Parametrização dos Schemas:** Incluir o campo agravo: `ArbovirusEnum = ArbovirusEnum.DENGUE` nos filtros de requisição e nas respostas de KPIs, séries temporais e alertas.
- **Fase 3 — Refatoração do MockDataGenerator:** Adaptar o gerador para calibrar os volumes de casos e curvas sazonais (ex.: Chikungunya com pico epidêmico em 2016-2017 e volume menor que dengue; Zika com pico em 2015-2016).
- **Fase 4 — Ajuste dos Repositórios e Endpoints:** Separar os dados em memória por código de agravo e receber agravo como query param em todos os endpoints REST.
- **Fase 5 — Atualização do Frontend:** Incluir seletor visual de doença na `FilterBarComponent` com chaveamento de cores (Dengue: Vermelho, Chikungunya: Laranja, Zika: Roxo, Febre Amarela: Amarelo) e propagação via `FilterStateService`.
- **Fase 6 — Banco Dimensional e ETL:** Inserir a tabela `dim_agravo` no schema PostgreSQL e traduzir o script de extração FTP do DataSUS para processar os arquivos DBC de cada arbovirose separadamente.

7. Fluxo de Execução do Sistema End-to-End

1. **Interação do Usuário:** Na `FilterBarComponent`, o usuário seleciona a UF (ex: 'MG'), o intervalo de anos (2020-2024) e a Arbovirose.
2. **Disparo Reativo:** O `FilterStateService` emite o novo estado via `BehaviorSubject`.
3. **Orquestração no Dashboard:** O `DashboardComponent` intercepta o evento e dispara 5 chamadas HTTP assíncronas em paralelo através do `EpidemiologyApiService`.
4. **Roteamento e Serviços Backend:** Os endpoints FastAPI recebem os parâmetros, acionam a camada de Services, que por sua vez consultam o `MockRepository` (ou banco PostgreSQL futuro).
5. **Processamento Matemático:** Os serviços agregam os dados populacionais, calculam taxas de incidência por 100k, quartis Q1/Mediana/Q3 do canal endêmico e avaliam o status de alerta.
6. **Renderização no Frontend:** As respostas tipadas atualizam os cards de KPI, o mapa Leaflet, os gráficos temporais e pirâmides etárias com feedback visual instantâneo.

Documentação gerada automaticamente para o projeto GAARA-II — Laboratório de Vigilância Epidemiológica Computacional.